WEB APPLICATION DEVELOPMENT
LAB 7 – INCLASS

I. User clicks "View Products":

Code:

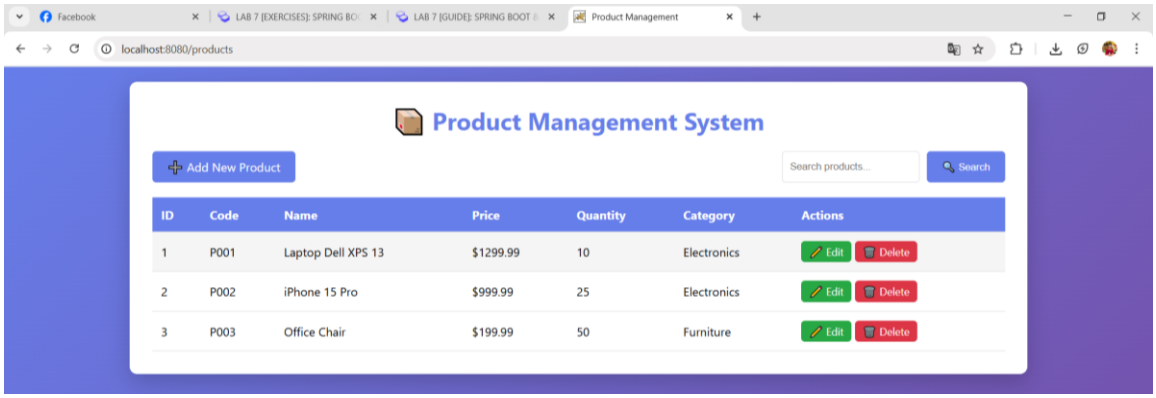
1. In ProductController.java

```
// List all products
@GetMapping
public String listProducts(Model model) {
    List<Product> products = productService.getAllProducts();
    model.addAttribute("products", products);
    return "product-list";  // Returns product-list.html
}
```

2. In ProductServiceImpl.java

```
@Override
public List<Product> getAllProducts() {
    return productRepository.findAll();
}
```

Output:



Display the list of products

Explanation about the code flow of view the list of products (example)

1. Browser → GET /products
2. ProductController receives request (@GetMapping)
3. Controller calls productService.getAllProducts()
4. Service calls productRepository.findAll()
5. JPA automatically queries database
6. Repository returns List<Product>
7. Controller adds data to Model
8. Returns view name "product-list"
9. Thymeleaf renders HTML
10. Response sent to browser

II. User clicks “Add New Product”:

Code:

1. In ProductController.java

```
// Show form for new product
@GetMapping("/new")
public String showNewForm(Model model) {
    Product product = new Product();
    model.addAttribute("product", product);
    return "product-form";
}
```

Show the add form when user clicks add button

```java
// Save product (create or update)
@PostMapping("/save")
public String saveProduct(@ModelAttribute("product") Product product, RedirectAttributes redirectAttributes) {
    try {
        productService.saveProduct(product);
        redirectAttributes.addFlashAttribute("message",
                product.getId() == null ? "Product added successfully!" : "Product updated successfully!");
    } catch (Exception e) {
        redirectAttributes.addFlashAttribute("error", "Error saving product: " + e.getMessage());
    }
    return "redirect:/products";
}
```

When the user click save button in add new product form

2. In ProductServiceImpl.java

```java
@Override
public Product saveProduct(Product product) {
    // Validation logic can go here
    return productRepository.save(product);
}
```

3. In product-list.html

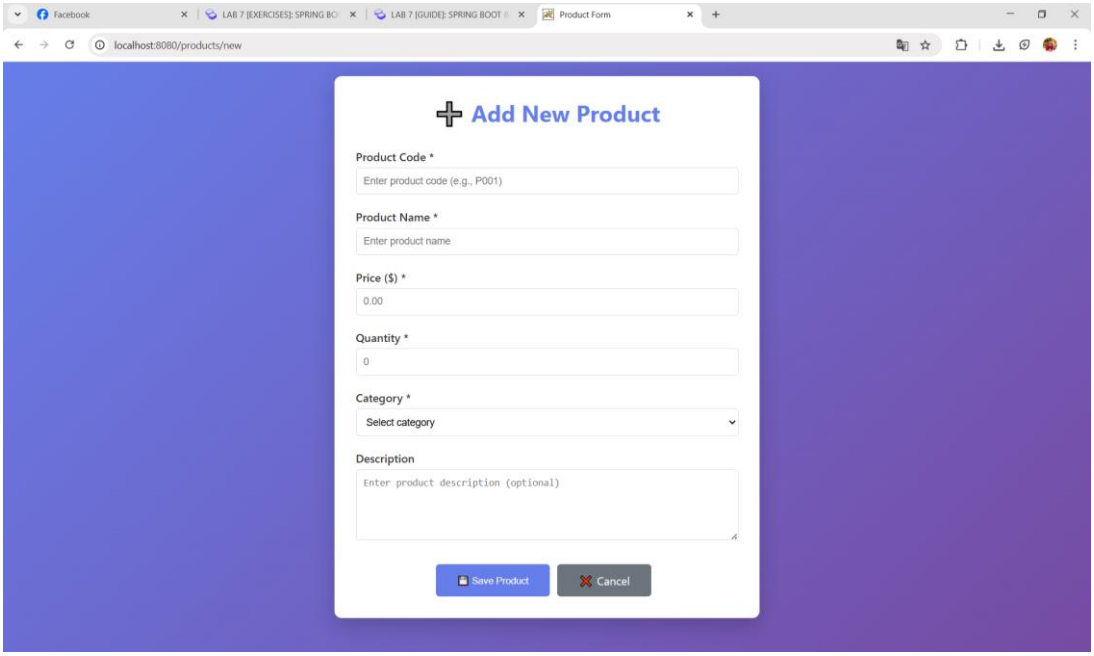
```html
<!-- Actions -->
<div class="actions">
    <a th:href="@{/products/new}" class="btn btn-primary">➕ Add New Product</a>
```

4. In product-form.html

```html
<div class="container">
    <h1 th:text="${product.id != null} ? '✏️ Edit Product' : '➕ Add New Product'">Product Form</h1>

    <form th:action="@{/products/save}" th:object="${product}" method="post">
        <!-- Hidden ID field for updates -->
        <input type="hidden" th:field="*{id}" />
```

Output:



Add new product form

Explanation about the code flow of the add new product

1. Browser → GET /products/new

– User clicks the “Add New Product” button which has this link:
<a th:href="@{/products/new}" ...>

2. ProductController receives request (@GetMapping("/new"))

– Spring MVC routes the request to:
@GetMapping("/new")
public String showNewForm(Model model)

3. Controller creates an empty Product object: Product product = new Product();
4. Controller adds empty product to Model: This allows Thymeleaf to bind form fields: model.addAttribute("product", product);
5. Controller returns view name "product-form": return "product-form";
6. Thymeleaf renders product-form.html: Form is displayed with empty fields: productCode, name, price, quantity, category, description.
7. Browser shows the Add Product form to the user then fills form and clicks "Save Product"
8. Browser → POST /products/save: The form sends data: <form th:action="@{/products/save}" method="post">
9. ProductController receives request (@PostMapping("/save")) public String saveProduct(@ModelAttribute("product") Product product)
10. Spring binds form inputs → Product object. It sets productCode, name, price, quantity, category, description.
11. Controller calls productService.saveProduct(product): productService.saveProduct(product);.
12. Service calls productRepository.save(product) JPA handles:

– INSERT if product.id == null (new)
– UPDATE if product.id exists

13. JPA writes new product into database: Hibernate generates SQL like insert into product (...)
14. Repository returns saved Product. Now product has generated ID.
15. Controller adds flash success message: redirectAttributes.addFlashAttribute("message", "Product added successfully!");.
16. Controller redirects user → /products: return "redirect:/products";.
17. Browser → GET /products again: This triggers the same flow as your "view product list".
18. User sees product-list with the new product visible

III. User clicks edit button

Code:

1. **In ProductController.java**

```
// Show form for editing product
@GetMapping("/edit/{id}")
public String showEditForm(@PathVariable Long id, Model model, RedirectAttributes redirectAttributes) {
    return productService.getProductById(id)
            .map(product -> {
                model.addAttribute("product", product);
                return "product-form";
            })
            .orElseGet(() -> {
                redirectAttributes.addFlashAttribute("error", "Product not found");
                return "redirect:/products";
            });
}
```

Show the add form when user clicks add button

```
// Save product (create or update)
@PostMapping("/save")
public String saveProduct(@ModelAttribute("product") Product product, RedirectAttributes redirectAttributes) {
    try {
        productService.saveProduct(product);
        redirectAttributes.addFlashAttribute("message",
                product.getId() == null ? "Product added successfully!" : "Product updated successfully!");
    } catch (Exception e) {
        redirectAttributes.addFlashAttribute("error", "Error saving product: " + e.getMessage());
    }
    return "redirect:/products";
}
```

When the user click save button in add new product form

2. **In ProductServiceImpl.java**

```java
@Override
public Product saveProduct(Product product) {
    // Validation logic can go here
    return productRepository.save(product);
}
```

3. **In product-list.html**

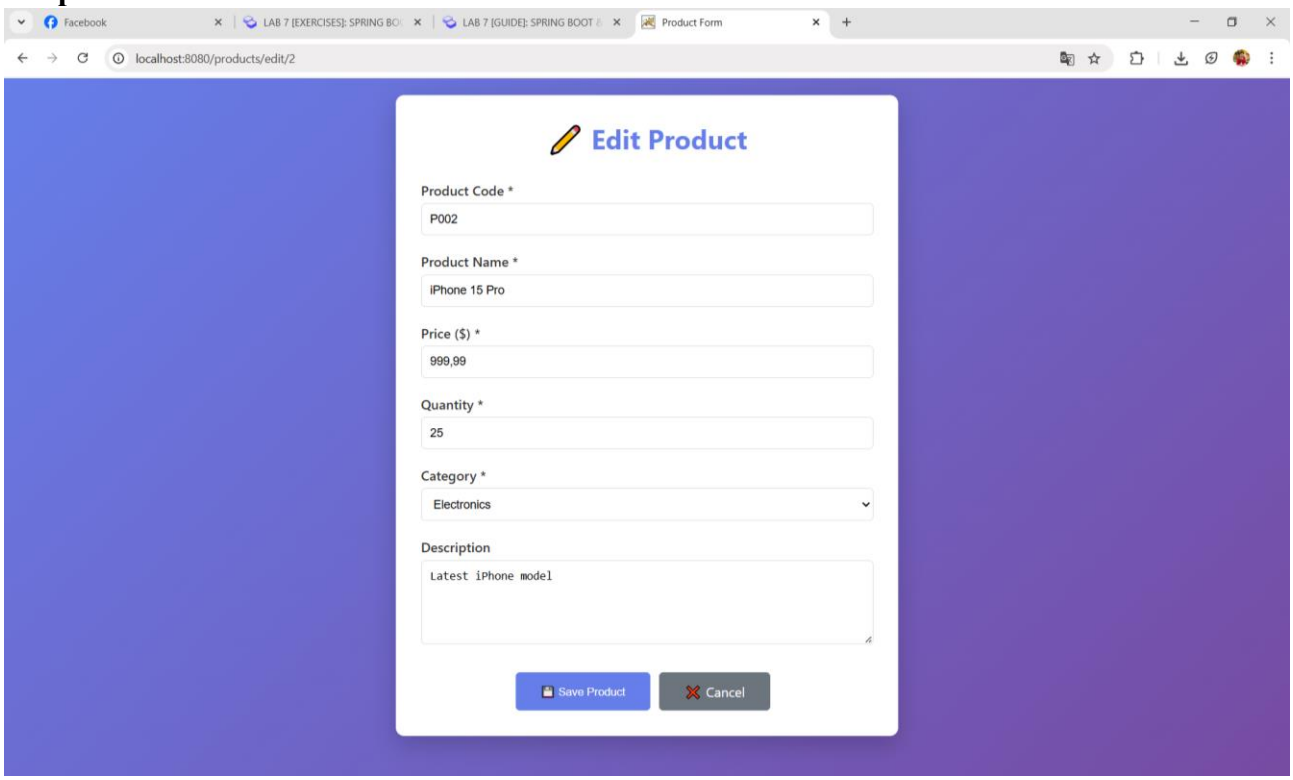
```html
<div class="actions-column">
    <a th:href="@{/products/edit/{id}(id=${product.id})}" class="btn btn-success btn-sm">✏️ Edit</a>
```

4. **In product-form.html**

```html
<div class="container">
    <h1 th:text="${product.id != null} ? '✏️ Edit Product' : '➕ Add New Product'">Product Form</h1>

    <form th:action="@{/products/save}" th:object="${product}" method="post">
        <!-- Hidden ID field for updates -->
        <input type="hidden" th:field="*{id}" />
```

Output:



Edit product form

Explanation about the code flow of the edited product:

1. Browser → GET /products/edit/{id} such as /products/edit/5. The link in the HTML is: <a th:href="@{/products/edit/{id}(id=${product.id})}">.
2. ProductController receives request (@GetMapping("/edit/{id}"))
   @GetMapping("/edit/{id}")
   public String showEditForm(@PathVariable Long id, Model model, RedirectAttributes redirectAttributes)
3. Controller calls productService.getProductById(id): productService.getProductById(id)
4. Service calls productRepository.findById(id). JPA automatically performs: select * from product where id = ?
5. Repository returns Optional<Product>
6. If product is found, controller adds product to Model: model.addAttribute("product", product);. Controller returns view "product-form": return "product-form";. Thymeleaf renders the form with existing product values. Form fields are pre-filled: productCode, name, price, quantity, category, description. Browser displays Edit Form to the user.
7. If product is not found, controller adds a flash error message:

redirectAttributes.addFlashAttribute("error", "Product not found"); and redirects to /product: return "redirect:/products"; and browser → GET /products. User updates fields and submits

8. Browser → POST /products/save. This is the same endpoint as Add Product.
9. Spring binds form inputs to Product object. This time product.id is **NOT null**, so this is an update.
10. Controller calls productService.saveProduct(product): productService.saveProduct(product);
11. Service calls productRepository.save(product). Because product.id exists → JPA performs UPDATE: update product set ... where id = ?
12. Repository returns updated Product
13. Controller adds flash message: redirectAttributes.addFlashAttribute("message", "Product updated successfully!");
14. Controller redirects → /products: return "redirect:/products";
15. Browser → GET /products. Product list reloads with success message.
16. User sees the updated product in the list.

IV. User clicks search button with given keyword

Code:

1. In ProductController.java

```
// Search products
@GetMapping("/search")
public String searchProducts(@RequestParam("keyword") String keyword, Model model) {
    List<Product> products = productService.searchProducts(keyword);
    model.addAttribute("products", products);
    model.addAttribute("keyword", keyword);
    return "product-list";
}
```

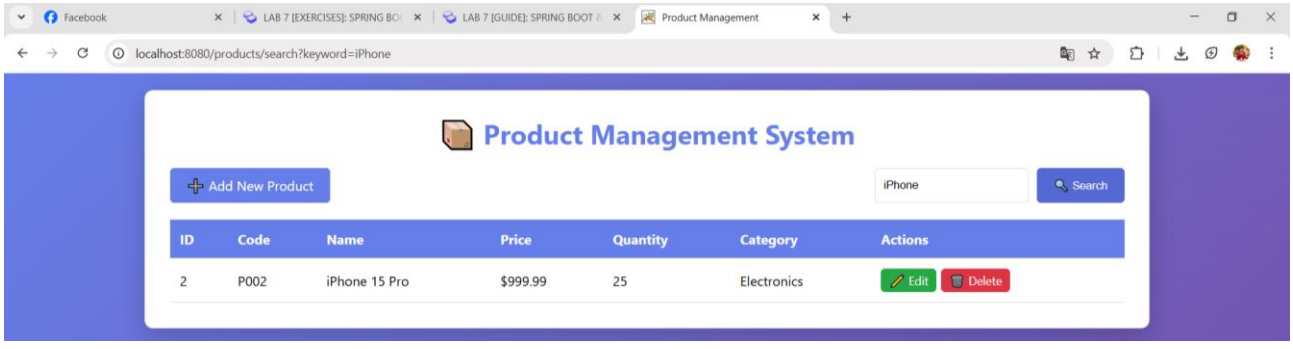
2. In ProductServiceImpl.java

```
@Override
public List<Product> searchProducts(String keyword) {
    return productRepository.findByNameContaining(keyword);
}
```

3. In product-list.html

```
<form th:action="@{/products/search}" method="get" class="search-form">
    <input type="text" name="keyword" th:value="${keyword}" placeholder="Search products..." />
    <button type="submit" class="btn btn-primary">🔍 Search</button>
</form>
```

Output:



Search term with “iPhone”

Explanation about the code flow of search button with given keyword:

1. Browser → GET /products/search?keyword=iPhone. The HTML form sends this request:

```
<form th:action="@{/products/search}" method="get" class="search-form">
    <input type="text" name="keyword" th:value="${keyword}" placeholder="Search products..." />
    <button type="submit" class="btn btn-primary">🔍 Search</button>
</form>
```

2. ProductController receives request (@GetMapping("/search"))
@GetMapping("/search")

public String searchProducts(@RequestParam("keyword") String keyword, Model model)

3. Spring injects the parameter “iPhone” into keyword. So, keyword = "iPhone";.
4. Controller calls productService.searchProducts(keyword): List<Product> products = productService.searchProducts("iPhone");.
5. Service calls productRepository.findByNameContaining(keyword): productRepository.findByNameContaining("iPhone");.

```
List<Product> findByNameContaining(String keyword);
```

6. Spring Data JPA creates SQL automatically.
7. Repository returns List<Product>.

– If matches found → list contains matching products.
– If no matches → list is empty.

8. Controller adds search results to Model: model.addAttribute("products", products);
9. Controller adds the search keyword back to the Model: model.addAttribute("keyword", keyword);.
10. Controller returns "product-list" view: return "product-list";.
11. Thymeleaf loads product-list.html.
12. If products list is NOT empty → Display matching rows in the table: <tr th:each="product : ${products}">.
13. If products list is empty → Display "No products found" message: .
14. Rendered HTML sent back to Browser.
15. Browser shows search results for “iPhone”.